

practical7

Q1_CreateFile.java

```
// Import BufferedWriter: used to write text to a file efficiently (buffered so writes are done in bulk).
import java.io.BufferedWriter;
// Import FileWriter: a character stream that opens a connection to a file and lets us write text into it.
import java.io.FileWriter;
// Import IOException: the exception type thrown when a file operation (open/write) fails.
import java.io.IOException;
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;

/**
 * Creates a new text file "JavaFile1.txt" and writes the text
 * entered from the console into it.
 */
public class Q1_CreateFile {
    public static void main(String[] args) {
        // Create a Scanner object that reads from System.in (the keyboard).
        Scanner sc = new Scanner(System.in);

        // Print a prompt telling the user what to enter.
        System.out.println("Enter the text to save in JavaFile1.txt (press Enter):");
        // Read the whole line typed by the user and store it in the String variable 'text'.
        String text = sc.nextLine();

        // try-with-resources: opens JavaFile1.txt for writing (overwrites existing content) and
        // wraps the FileWriter in a BufferedWriter for efficiency. The writer auto-closes at the end of this
        block.
        try (BufferedWriter writer = new BufferedWriter(new FileWriter("JavaFile1.txt"))) {
            // Write the user-entered text into the file.
            writer.write(text);
            // Print a confirmation message to the console.
            System.out.println("Text saved successfully to JavaFile1.txt");
        } catch (IOException e) {
            // If opening/writing the file fails, catch the error here.
            System.out.println("Error writing to file: " + e.getMessage());
        }
        // End of the try-catch block. The writer was closed automatically.
    }
}
```

Q2_CopyFile.java

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

/**
 * Reads the content of "JavaFile1.txt" and copies it to
 * "JavaFile2.txt" using file streams.
 */
public class Q2_CopyFile {
    public static void main(String[] args) {
        try (FileInputStream in = new FileInputStream("JavaFile1.txt");
            FileOutputStream out = new FileOutputStream("JavaFile2.txt")) {

            int ch;
            while ((ch = in.read()) != -1) {
                out.write(ch);
            }
            System.out.println("Content copied from JavaFile1.txt to JavaFile2.txt");
        } catch (IOException e) {
            System.out.println("Error copying file: " + e.getMessage());
        }
    }
}
```

Q3_CountStats.java

```
// Import BufferedReader: reads a file line-by-line efficiently.
import java.io.BufferedReader;
// Import FileReader: a character stream that opens a connection to a file for reading.
import java.io.FileReader;
// Import IOException: the exception type thrown when file reading fails.
import java.io.IOException;

/**
 * Counts the number of lines, words and characters in a given text file.
 * Pass the filename as a command-line argument or uses "JavaFile1.txt" by default.
 */
public class Q3_CountStats {
    public static void main(String[] args) {
        // The file to analyze. Hard-coded default (a command line arg could override it, but it isn't used here).
        String filename = "JavaFile1.txt";

        // Counter for the number of lines, started at 0.
        int lines = 0;
        // Counter for the number of words, started at 0.
        int words = 0;
        // Counter for the number of characters, started at 0.
        int characters = 0;

        // try-with-resources: open the file for reading and wrap it in a BufferedReader
        // so we can read whole lines. The reader auto-closes when the block ends.
        try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
            // Will hold one line of the file at a time.
            String line;

            // readLine() returns the next line, or null when there is no more data (end of file).
            // The loop keeps running while the returned value is NOT null.
            while ((line = reader.readLine()) != null) {
                // This line counts, so increment the line counter.
                lines++;
                // Add the length of the current line to the total character count.
                // (Note: this does NOT count the invisible newline characters between lines.)
                characters += line.length();
                // Split the trimmed line on one-or-more whitespace characters (\s+) to get an array of words.
                // trim() removes leading/trailing spaces so we don't get empty tokens.
                String[] tokens = line.trim().split("\\s+");
                // Guard: only count words if the line is not blank (avoid counting empty lines as having a word).
                if (!line.trim().isEmpty()) {
                    // Add the number of words on this line to the total word count.
                    words += tokens.length;
                }
            }
            // End of the while loop; we have finished reading every line.

            // Print the name of the file that was analyzed.
            System.out.println("File: " + filename);
            // Print the total number of lines found.
            System.out.println("Number of Lines: " + lines);
            // Print the total number of words found.
            System.out.println("Number of Words: " + words);
            // Print the total number of characters found.
            System.out.println("Number of Characters: " + characters);
        } catch (IOException e) {
            // If opening or reading the file fails, catch the error here.
            System.out.println("Error reading file: " + e.getMessage());
        }
    }
}
```

```
    }  
    // End of try-catch. The reader was closed automatically.  
  }  
}
```

Q4_SearchWord.java

```
// Import BufferedReader: reads a file line-by-line efficiently.
import java.io.BufferedReader;
// Import FileReader: a character stream that opens a connection to a file for reading.
import java.io.FileReader;
// Import IOException: the exception type thrown when file reading fails.
import java.io.IOException;
// Import Scanner: used to read the search word typed on the keyboard.
import java.util.Scanner;

/**
 * Searches for a specific word in a file and prints the number
 * of occurrences along with the line numbers where it appears.
 */
public class Q4_SearchWord {
    public static void main(String[] args) {
        // Create a Scanner object that reads from the keyboard.
        Scanner sc = new Scanner(System.in);
        // The file to search inside. Hard-coded default.
        String filename = "JavaFile1.txt";

        // Print a prompt (no newline) asking the user for the word to find.
        System.out.print("Enter the word to search for: ");
        // Read the single word typed by the user (stops at the first space or Enter).
        String searchWord = sc.next();

        // Total number of times the word appears, started at 0.
        int count = 0;
        // Tracks which line of the file we are currently on (for reporting). Starts at 0.
        int lineNo = 0;

        // try-with-resources: open the file for reading wrapped in a BufferedReader.
        // The reader auto-closes when the block ends.
        try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
            // Will hold one line of the file at a time.
            String line;

            // readLine() returns the next line, or null at the end of the file. Loop until end.
            while ((line = reader.readLine()) != null) {
                // Move to the next line number (1-based line numbering).
                lineNo++;
                // Split the trimmed line on whitespace to get an array of individual words.
                String[] words = line.trim().split("\\s+");
                // Loop over every word in this line.
                for (String word : words) {
                    // Compare the current word with the search word, ignoring letter case
                    // (e.g. "Hello" and "hello" are treated as equal).
                    if (word.equalsIgnoreCase(searchWord)) {
                        // Match found: increment the occurrence counter.
                        count++;
                        // Print the line number and the trimmed contents of the line where it was found.
                        System.out.println("Found on line " + lineNo + ": \"" + line.trim() + "\"");
                    }
                }
                // End of the per-word loop.
            }
            // End of the while loop; the whole file has been searched.

            // Print the final total number of occurrences with a blank line before it.
            System.out.println("\nTotal occurrences of \"" + searchWord + "\": " + count);
        }
    }
}
```

```
    } catch (IOException e) {  
        // If opening or reading the file fails, catch the error here.  
        System.out.println("Error reading file: " + e.getMessage());  
    }  
    // End of try-catch. The reader was closed automatically.  
}
```

Q5_AWTForm.java

```
// Import Button: an AWT clickable button component.
import java.awt.Button;
// Import Frame: the top-level AWT window that holds other components.
import java.awt.Frame;
// Import Label: a component that displays static text on the form.
import java.awt.Label;
// Import Panel: a container used to group other components together.
import java.awt.Panel;
// Import TextField: a single-line text input box.
import java.awt.TextField;
// Import GridBagLayout: a flexible grid-based layout manager for placing components precisely.
import java.awt.GridBagLayout;
// Import GridBagConstraints: sets position/size/alignment of each component placed in the GridBagLayout.
import java.awt.GridBagConstraints;
// Import Insets: defines the spacing (padding) around each component.
import java.awt.Insets;

/**
 * AWT Login Form layout (design only - no event handling).
 * Practical 8 adds event handling to this form.
 */
public class Q5_AWTForm {
    public static void main(String[] args) {
        // Create the main window and give it the title "Login Form".
        Frame frame = new Frame("Login Form");
        // Set the window width to 400 and height to 300 pixels.
        frame.setSize(400, 300);
        // Set the layout manager to GridBagLayout for precise component placement.
        frame.setLayout(new GridBagLayout());
        // Create the constraints object that will control where each component is placed.
        GridBagConstraints gbc = new GridBagConstraints();
        // Give each component 10 pixels of padding on all four sides (top, left, bottom, right).
        gbc.insets = new Insets(10, 10, 10, 10);

        // Title: put the next component in column 0, row 0.
        gbc.gridx = 0;
        gbc.gridy = 0;
        // Let this title span two columns so it can be centered across the form.
        gbc.gridwidth = 2;
        // Add the title label "--- Login Form ---" using the current constraints.
        frame.add(new Label("--- Login Form ---"), gbc);

        // Gmail ID label: move to row 1 (still column 0).
        gbc.gridy = 1;
        // Reset the span back to one column.
        gbc.gridwidth = 1;
        // Align this label to the EAST (right side) of its cell.
        gbc.anchor = GridBagConstraints.EAST;
        // Add the "Gmail ID:" label at this position.
        frame.add(new Label("Gmail ID:"), gbc);

        // Create a text field that can hold roughly 20 characters of input; this stores the Gmail.
        TextField gmailField = new TextField(20);
        // Move to column 1 (same row 1) for the input field.
        gbc.gridx = 1;
        // Align this field to the WEST (left side) of its cell.
        gbc.anchor = GridBagConstraints.WEST;
        // Add the Gmail input field next to its label.
        frame.add(gmailField, gbc);
    }
}
```

```

// Password label: move back to column 0, then down to row 2.
gbc.gridx = 0;
gbc.gridy = 2;
// Align the label to the right (EAST) of its cell.
gbc.anchor = GridBagConstraints.EAST;
// Add the "Password:" label at this position.
frame.add(new Label("Password:"), gbc);

// Create a text field for the password input.
TextField passwordField = new TextField(20);
// Mask every typed character with '*' so the password is hidden as you type.
passwordField.setEchoChar('*');
// Move to column 1 (same row 2) for the password field.
gbc.gridx = 1;
// Align the field to the left (WEST) of its cell.
gbc.anchor = GridBagConstraints.WEST;
// Add the password input field next to its label.
frame.add(passwordField, gbc);

// Create a Panel container to hold the buttons side by side.
Panel buttonPanel = new Panel();
// Add a "Login" button to the panel.
buttonPanel.add(new Button("Login"));
// Add a "Reset" button to the panel.
buttonPanel.add(new Button("Reset"));

// Place the button panel at column 0, row 3.
gbc.gridx = 0;
gbc.gridy = 3;
// Make the panel span both columns so the buttons sit centered below the fields.
gbc.gridwidth = 2;
// Add the button panel to the frame.
frame.add(buttonPanel, gbc);

// Center the whole window in the middle of the screen.
frame.setLocationRelativeTo(null);
// Make the window visible to the user.
frame.setVisible(true);
// End of main. Note: no event handling here, so clicking buttons does nothing yet.

```

```

}

```

```

}

```

Q5_b_Calculator.java

```
// Import Button: an AWT clickable button component.
import java.awt.Button;
// Import Color: defines custom foreground and background colors.
import java.awt.Color;
// Import ComponentOrientation: allows setting right-to-left orientation for display alignment.
import java.awt.ComponentOrientation;
// Import Font: defines font family, style (bold/plain), and size.
import java.awt.Font;
// Import Frame: the top-level AWT window that holds other components.
import java.awt.Frame;
// Import GridLayout: a layout manager that arranges components in a rectangular grid of equal cells.
import java.awt.GridLayout;
// Import Panel: a lightweight container used to group components together.
import java.awt.Panel;
// Import TextField: a single-line text input/display component.
import java.awt.TextField;

/**
 * AWT Calculator (GUI design only - no event handling).
 * Practical 8 adds event handling to this form.
 *
 * Layout:
 * - Dark slate frame background
 * - Top display screen showing "0"
 * - 4x4 button grid:
 *   [ 7 ] [ 8 ] [ 9 ] [ ÷ ]
 *   [ 4 ] [ 5 ] [ 6 ] [ x ]
 *   [ 1 ] [ 2 ] [ 3 ] [ - ]
 *   [ 0 ] [ . ] [ + ] [ = ]
 * - Crisp white standard buttons with bold dark text
 * - High-visibility orange "=" button
 */
public class Q5_b_Calculator {
    public static void main(String[] args) {
        System.out.println("Starting Practical 7 - Question 5.b: AWT Calculator (design only)...");

        // Create the main window frame with title "Calculator".
        Frame frame = new Frame("Calculator");
        // Set the window width to 320 and height to 440 pixels.
        frame.setSize(320, 440);
        // Use absolute layout so each component is positioned by exact coordinates.
        frame.setLayout(null);
        // Set the dark gray/slate background color matching the design.
        frame.setBackground(new Color(68, 73, 80));
        // Prevent the user from resizing the window.
        frame.setResizable(false);

        // Create the display screen at the top, starting with the value "0".
        TextField display = new TextField("0");
        // Place the display at x=20, y=45 with width 280 and height 55.
        display.setBounds(20, 45, 280, 55);
        // Prevent the user from typing directly in the display.
        display.setEditable(false);
        // Give the display a light grayish-blue background matching the design.
        display.setBackground(new Color(220, 226, 230));
        // Use dark text so the display value is easy to read.
        display.setForeground(new Color(25, 25, 25));
        // Use a large bold monospaced font to mimic a digital calculator screen.
        display.setFont(new Font("Monospaced", Font.BOLD, 28));
```

```

// Right-align the text by using a right-to-left component orientation.
display.setComponentOrientation(ComponentOrientation.RIGHT_TO_LEFT);
// Add the display to the frame.
frame.add(display);

// Create a Panel that will hold the 4x4 button grid.
Panel buttonPanel = new Panel();
// Place the panel at x=20, y=115 with width 280 and height 290.
buttonPanel.setBounds(20, 115, 280, 290);
// Arrange 4 rows and 4 columns with 10px horizontal and vertical gaps.
buttonPanel.setLayout(new GridLayout(4, 4, 10, 10));
// Match the panel background to the dark frame background.
buttonPanel.setBackground(new Color(68, 73, 80));

// Array of button labels matching the 4x4 layout from the assignment image.
String[] buttons = {
    "7", "8", "9", "÷",
    "4", "5", "6", "x",
    "1", "2", "3", "-",
    "0", ".", "+", "="
};

// Create, style, and add each button one at a time.
for (String text : buttons) {
    // Create the button with its label text.
    Button btn = new Button(text);
    // Use a large bold sans-serif font so the buttons are easy to read.
    btn.setFont(new Font("SansSerif", Font.BOLD, 22));

    // Give the "=" button orange styling; all others get white styling.
    if (text.equals("=")) {
        btn.setBackground(new Color(245, 142, 44)); // Vibrant calculator orange
        btn.setForeground(Color.WHITE);
    } else {
        btn.setBackground(Color.WHITE);
        btn.setForeground(new Color(35, 35, 35));
    }

    // Add the styled button to the grid panel.
    buttonPanel.add(btn);
}

// Add the button grid panel to the frame.
frame.add(buttonPanel);

// Center the whole window in the middle of the screen.
frame.setLocationRelativeTo(null);
// Make the window visible to the user.
frame.setVisible(true);
// End of main. Note: no event handling here, so clicking buttons does nothing yet.
}
}

```

Q5_c_RegistrationForm.java

```
// Import Button: an AWT clickable button component.
import java.awt.Button;
// Import Checkbox: creates checkboxes or radio buttons (when grouped).
import java.awt.Checkbox;
// Import CheckboxGroup: groups checkboxes together into mutually exclusive radio buttons.
import java.awt.CheckboxGroup;
// Import Choice: creates a drop-down selection menu.
import java.awt.Choice;
// Import Color: defines custom foreground and background colors.
import java.awt.Color;
// Import FlowLayout: arranges components horizontally in a line.
import java.awt.FlowLayout;
// Import Font: defines font family, style (bold/plain), and size.
import java.awt.Font;
// Import Frame: the top-level AWT window that holds other components.
import java.awt.Frame;
// Import GridBagConstraints: sets position/size/alignment of each component placed in a GridBagLayout.
import java.awt.GridBagConstraints;
// Import GridBagLayout: a flexible grid-based layout manager for placing components precisely.
import java.awt.GridBagLayout;
// Import Insets: defines the spacing (padding) around each component.
import java.awt.Insets;
// Import Label: a component that displays static text on the form.
import java.awt.Label;
// Import Panel: a lightweight container used to group components together.
import java.awt.Panel;
// Import TextArea: a multi-line text input component.
import java.awt.TextArea;
// Import TextField: a single-line text input box.
import java.awt.TextField;

/**
 * AWT Registration Form (GUI design only - no event handling).
 * Practical 8 adds event handling to this form.
 *
 * Layout:
 * - Dark slate-blue window background
 * - Bright yellow "Registration Form" title header
 * - Fields:
 *   - Name: TextField (prefilled with "Ram")
 *   - Father Name: TextField (prefilled with "Kumar")
 *   - Age: TextField (prefilled with "23")
 *   - Gender: Radio buttons (Male selected, Female)
 *   - Course: Drop-down Choice menu (preselected "Java")
 *   - Hobbies: Multiple Checkboxes (Drawing selected, Singing, Music, Others)
 *   - Address: Multi-line TextArea with prefilled address
 * - "Save Details" button with vibrant blue background
 * - "Clear All" button with vibrant red background
 */
public class Q5_c_RegistrationForm {
    public static void main(String[] args) {
        System.out.println("Starting Practical 7 - Question 5.c: AWT Registration Form (design only)...");

        // Create the main window frame with title "Registration Form".
        Frame frame = new Frame("Registration Form");
        // Set the window width to 520 and height to 580 pixels.
        frame.setSize(520, 580);
        // Set the layout manager to GridBagLayout for precise component placement.
        frame.setLayout(new GridBagLayout());
    }
}
```

```

// Use a dark slate-blue background color matching the design.
Color bgColor = new Color(50, 56, 68);
frame.setBackground(bgColor);

// Create the fonts used for the title, labels, inputs, and buttons.
Font titleFont = new Font("SansSerif", Font.BOLD, 20);
Font labelFont = new Font("SansSerif", Font.BOLD, 14);
Font inputFont = new Font("SansSerif", Font.PLAIN, 13);
Font buttonFont = new Font("SansSerif", Font.BOLD, 13);

// Create the constraints object that will control where each component is placed.
GridBagConstraints gbc = new GridBagConstraints();
// Give each component padding around all four sides, and stretch it horizontally.
gbc.insets = new Insets(6, 12, 6, 12);
gbc.fill = GridBagConstraints.HORIZONTAL;

// Row 0: Yellow "Registration Form" Title.
gbc.gridx = 0;
gbc.gridy = 0;
// Let this title span two columns so it sits across the whole form.
gbc.gridwidth = 2;
gbc.anchor = GridBagConstraints.WEST;
Label titleLabel = new Label("Registration Form");
titleLabel.setFont(titleFont);
titleLabel.setForeground(Color.YELLOW);
titleLabel.setBackground(bgColor);
frame.add(titleLabel, gbc);

// Row 1: Name Field.
gbc.gridy = 1;
// Reset the span back to one column.
gbc.gridwidth = 1;
gbc.anchor = GridBagConstraints.WEST;
Label nameLbl = new Label("Name");
nameLbl.setFont(labelFont);
nameLbl.setForeground(Color.WHITE);
nameLbl.setBackground(bgColor);
frame.add(nameLbl, gbc);

gbc.gridx = 1;
// Create a text field prefilled with "Ram".
TextField nameField = new TextField("Ram", 25);
nameField.setFont(inputFont);
frame.add(nameField, gbc);

// Row 2: Father Name Field.
gbc.gridx = 0;
gbc.gridy = 2;
Label fatherNameLbl = new Label("Father Name");
fatherNameLbl.setFont(labelFont);
fatherNameLbl.setForeground(Color.WHITE);
fatherNameLbl.setBackground(bgColor);
frame.add(fatherNameLbl, gbc);

gbc.gridx = 1;
// Create a text field prefilled with "Kumar".
TextField fatherNameField = new TextField("Kumar", 25);
fatherNameField.setFont(inputFont);
frame.add(fatherNameField, gbc);

// Row 3: Age Field.
gbc.gridx = 0;
gbc.gridy = 3;

```

```

Label ageLbl = new Label("Age");
ageLbl.setFont(labelFont);
ageLbl.setForeground(Color.WHITE);
ageLbl.setBackground(bgColor);
frame.add(ageLbl, gbc);

gbc.gridx = 1;
// Create a text field prefilled with "23".
TextField ageField = new TextField("23", 25);
ageField.setFont(inputFont);
frame.add(ageField, gbc);

// Row 4: Gender Radio Buttons (CheckboxGroup).
gbc.gridx = 0;
gbc.gridy = 4;
Label genderLbl = new Label("Gender");
genderLbl.setFont(labelFont);
genderLbl.setForeground(Color.WHITE);
genderLbl.setBackground(bgColor);
frame.add(genderLbl, gbc);

gbc.gridx = 1;
// Create a Panel to hold the radio buttons side by side.
Panel genderPanel = new Panel(new FlowLayout(FlowLayout.LEFT, 10, 0));
genderPanel.setBackground(bgColor);
// Group the two radio buttons so only one can be selected at a time.
CheckboxGroup genderGroup = new CheckboxGroup();
Checkbox maleRadio = new Checkbox("Male", genderGroup, true);
Checkbox femaleRadio = new Checkbox("Female", genderGroup, false);
maleRadio.setFont(inputFont);
maleRadio.setForeground(Color.WHITE);
maleRadio.setBackground(bgColor);
femaleRadio.setFont(inputFont);
femaleRadio.setForeground(Color.WHITE);
femaleRadio.setBackground(bgColor);
genderPanel.add(maleRadio);
genderPanel.add(femaleRadio);
frame.add(genderPanel, gbc);

// Row 5: Course Dropdown (Choice).
gbc.gridx = 0;
gbc.gridy = 5;
Label courseLbl = new Label("Course");
courseLbl.setFont(labelFont);
courseLbl.setForeground(Color.WHITE);
courseLbl.setBackground(bgColor);
frame.add(courseLbl, gbc);

gbc.gridx = 1;
// Create a drop-down menu with a list of courses, preselected to "Java".
Choice courseChoice = new Choice();
courseChoice.setFont(inputFont);
courseChoice.add("Java");
courseChoice.add("Python");
courseChoice.add("C++");
courseChoice.add("Web Development");
courseChoice.add("Data Science");
courseChoice.select("Java");
frame.add(courseChoice, gbc);

// Row 6: Hobbies Checkboxes.
gbc.gridx = 0;
gbc.gridy = 6;

```

```

Label hobbiesLbl = new Label("Hobbies");
hobbiesLbl.setFont(labelFont);
hobbiesLbl.setForeground(Color.WHITE);
hobbiesLbl.setBackground(bgColor);
frame.add(hobbiesLbl, gbc);

gbc.gridx = 1;
// Create a Panel to hold the hobby checkboxes side by side.
Panel hobbiesPanel = new Panel(new FlowLayout(FlowLayout.LEFT, 8, 0));
hobbiesPanel.setBackground(bgColor);
// Create the four hobby checkboxes; only "Drawing" starts selected.
Checkbox cbDrawing = new Checkbox("Drawing", true);
Checkbox cbSinging = new Checkbox("Singing", false);
Checkbox cbMusic = new Checkbox("Music", false);
Checkbox cbOthers = new Checkbox("Others", false);
cbDrawing.setFont(inputFont);
cbDrawing.setForeground(Color.WHITE);
cbDrawing.setBackground(bgColor);
cbSinging.setFont(inputFont);
cbSinging.setForeground(Color.WHITE);
cbSinging.setBackground(bgColor);
cbMusic.setFont(inputFont);
cbMusic.setForeground(Color.WHITE);
cbMusic.setBackground(bgColor);
cbOthers.setFont(inputFont);
cbOthers.setForeground(Color.WHITE);
cbOthers.setBackground(bgColor);
hobbiesPanel.add(cbDrawing);
hobbiesPanel.add(cbSinging);
hobbiesPanel.add(cbMusic);
hobbiesPanel.add(cbOthers);
frame.add(hobbiesPanel, gbc);

// Row 7: Address TextArea.
gbc.gridx = 0;
gbc.gridy = 7;
// Align this label to the top-left corner of its cell.
gbc.anchor = GridBagConstraints.NORTHWEST;
Label addressLbl = new Label("Address");
addressLbl.setFont(labelFont);
addressLbl.setForeground(Color.WHITE);
addressLbl.setBackground(bgColor);
frame.add(addressLbl, gbc);

gbc.gridx = 1;
// Create a multi-line text area prefilled with the address and a vertical scrollbar.
TextArea addressArea = new TextArea("234 - 1d First Street,\nAnna Main Road\nNamakkal", 4, 25,
TextArea.SCROLLBARS_VERTICAL_ONLY);
addressArea.setFont(inputFont);
frame.add(addressArea, gbc);

// Row 8: Action Buttons (Save Details & Clear All).
gbc.gridx = 0;
gbc.gridy = 8;
// Let the buttons span both columns so they sit centered below the fields.
gbc.gridwidth = 2;
gbc.anchor = GridBagConstraints.CENTER;
gbc.insets = new Insets(16, 12, 12, 12);

// Create a Panel to hold the two buttons side by side.
Panel btnPanel = new Panel(new FlowLayout(FlowLayout.CENTER, 20, 0));
btnPanel.setBackground(bgColor);

```

```

// Create the blue "Save Details" button.
Button saveBtn = new Button("Save Details");
saveBtn.setFont(buttonFont);
saveBtn.setBackground(new Color(0, 50, 230));
saveBtn.setForeground(Color.WHITE);
btnPanel.add(saveBtn);

// Create the red "Clear All" button.
Button clearBtn = new Button("Clear All");
clearBtn.setFont(buttonFont);
clearBtn.setBackground(new Color(220, 20, 20));
clearBtn.setForeground(Color.WHITE);
btnPanel.add(clearBtn);

frame.add(btnPanel, gbc);

// Center the whole window in the middle of the screen.
frame.setLocationRelativeTo(null);
// Make the window visible to the user.
frame.setVisible(true);
// End of main. Note: no event handling here, so clicking buttons does nothing yet.
}
}

```